

# ESISAR - Examen IN331

## *Examen de Programmation C sur machine*

31 janvier 2025 - Durée 2h

### Recommandations Générales

#### Consignes

Aucun document n'est autorisé à l'exception de ceux fournis avec cet énoncé sous forme électronique :

1. Consignes préliminaires pour effectuer cet examen : *consignesEleves-amont.pdf*

Nous vous recommandons de suivre strictement les recommandations de ce document.

2. Rappels de la syntaxe du langage C : *CheatSheet-LangageC.pdf*

**La compréhension du sujet faisant partie de l'épreuve, il ne sera répondu à aucune question.**

S'il vous semble qu'une erreur s'est glissée dans le sujet, indiquez comment vous la corrigez et répondez à la question en en tenant compte.

#### Barème

Le barème est donné à titre indicatif. Il pourra évoluer à la marge.

#### Conseils pour réussir cet examen

- Commencer par vérifier que la commande `make` fonctionne correctement. Ça doit compiler dès le départ.

Faites en sorte de ne jamais trop vous éloigner d'un état dans lequel vous êtes en mesure de compiler. Attention : À la fin de l'épreuve, le projet rendu doit être "compilable" !

- Implémenter des blocs de code simples et tester tous les cas de figure possibles.

#### Fichiers fournis

La totalité des fichiers du projet à rendre se trouvent dans le fichier `tar.gz` ci-joint.

- Le fichier `readme.md` décrit en anglais l'énoncé des différents exercices à réaliser. La version en français est donnée dans la suite du présent document.
- Le fichier `Makefile` sert à gérer la construction du projet. Il contient les règles et les dépendances pour compiler vos fichiers sources et générer l'exécutable. Il permet de produire le fichier exécutable `out/bin/exam` ainsi que la bibliothèque `out/lib/libfile_metrics.so`. Le fichier `Makefile` est *pour lecture seule*. **Il ne doit pas être modifié !**
- Le fichier d'entête `inc/functions.h` contient la déclaration des fonctions à écrire, les macros et les définitions partagées par les différents fichiers C. Ce fichier est également *pour lecture seule*. **Il ne doit pas être modifié !**

- Il est demandé d'écrire votre code dans les fichiers `src/*.c`. Il s'agit de l'implémentation des fonctions déclarées dans le fichier d'entête `functions.h`. Les fichiers C sont fournis vides (signature seule sans implémentation). Il vous est demandé de modifier ces fichiers pour réaliser le projet.

Pour rappel, **seuls les fichiers C doivent être modifiés**. Le `Makefile` et les fichiers de header `*.h` doivent être conservés *en lecture seule* afin de garantir l'intégrité du projet.

## Sujet de l'examen

Pour cet examen, il vous est demandé d'implémenter les fonctions qui sont détaillées dans le fichier d'entête `inc/functions.h`. Vous y trouverez notamment la spécification des entrées et des sorties de chacune des fonctions attendues.

### Exercice 1 : Fibonacci (2 points)

Un petit exercice pour s'échauffer : écrire une fonction C qui calcule la valeur de Fibonacci pour un entier  $n$ .

Pour mémoire, la suite de Fibonacci se définit de la manière suivante :

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$  pour  $n > 1$

Fichiers à implémenter : `fibonacci.c`

### Exercice 2 : Analyse de fichier (4 points)

Dans cet exercice, vous devez analyser un fichier et retourner des métriques. Il vous est demandé de calculer à partir d'un fichier de données les métriques suivantes :

- la plus petite longueur d'une ligne,
- la plus grande longueur d'une ligne,
- la longueur totale de toutes les lignes,
- le nombre de lignes.

Votre implémentation doit être la plus efficace possible et doit prendre en compte les cas limites tels que les fichiers vides ou les fichiers ne contenant que des caractères espace.

Fichier à implémenter : `file_metrics.c`

### Exercice 3 : Chiffrement de César (6 points)

Dans cet exercice, vous devez implémenter une fonction permettant de déchiffrer un message encodé avec le chiffrement de César. Le chiffrement de César, aussi appelé *chiffrement par décalage* prend chaque lettre d'un texte et les décale d'un nombre fixe de positions dans l'alphabet.

Par exemple, avec un décalage de 3 "A" devient "D", "B" devient "E", et ainsi de suite. La fonction doit traiter aussi bien les majuscules que les minuscules, mais préserve les caractères non-alphabétiques.

## Contraintes

1. La fonction doit déchiffrer correctement un texte en décalant chaque lettre dans la direction

inverse de l'encodage.

2. Vous devez traiter aussi bien les lettres majuscules ("A-Z") que les minuscules ("a-z").
3. Les caractères non-alphabétiques doivent rester inchangés.

## Exemple

```
char message[] = "Khoor Zruog!";
int shift = 3;
caesarDecipher(message, shift);
printf("%s\n", message); // Output: "Hello World!"
```

Fichiers à implémenter `caesar.c`

## Exercice 4 : Unscrapper (8 points)

Pour cet exercice, vous disposez d'un fichier de données contenant une phrase clé qui a été découpée en morceaux. Chaque morceau est encadré par les chaînes de caractères "scr://" and ";".

Il vous est demandé d'extraire et de reconstruire la phrase clé à partir de ces segments.

Par exemple, si le fichier de données contient le texte suivant

```
random text scr://key; more random text scr://phrase; even more random text
```

alors la phrase clé (qui est encodée avec le chiffrement de César) à extraire est :

```
key phrase
```

Étapes pour la résolution de cet exercice :

1. Passer le nom du fichier en argument à la fonction `main`.
2. Lire le fichier de données.
3. Identifier et extraire tous les morceaux encadrés par "scr://" and ";".
4. Concatener les morceaux extraient afin de former la phrase clé.
5. Utiliser `caesarDecipher` pour obtenir le message en clair.

Le fichier de données qui vous est fourni se trouve `data/scrambled.dat`. (Indice important : le premier caractère non chiffré est un "s").

Fichiers à implémenter `unscrambler.c`

## Astuces

- La fonction `fgets` peut être utilisée pour lire une ligne dans le fichier de données. Elle lit jusqu'à ce qu'un caractère "NewLine" ou "End Of File" soit atteint. Vous êtes invités à consulter la page de man de `fgets`.
- Comme leur nom le laisse supposer, les fonctions `fopen` et `fclose` sont utilisées pour respectivement ouvrir ou fermer un fichier. Lorsque vous souhaitez ouvrir un fichier en *lecture*, utilisez le mode "r".

Voici un exemple :

```
FILE *file = fopen("filename.txt", "r");
if (file == NULL) {
    // Handle error
```

```
}  
// Perform file operations  
fclose(file);
```

Vous êtes invités à consulter la page de man de `fopen` et `fclose`. Vous y trouverez des façons de les utiliser ainsi que des exemples.